Self Balancing Robot

Autonomous Mobile Robotics- Course Project

Submitted in partial fulfillment of the requirements of
BITS F451 Course Project

By

Srujan Deolasee — ID No. 2019A7PS1139P

Sachin Thakur — 2018A4PS0114P

Under the supervision of:

Dr. Avinash Gautam
Dr. Sudeept Mohan

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI, PILANI CAMPUS

December 2021

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI, PILANI CAMPUS

Abstract

Self Balancing Robot

by Srujan Deolasee Sachin Thakur

A strictly two-wheeled robot (without a free castor wheel for support) provides a helpful platform for testing complex control algorithms. It helps ensure if a multi-variable system will be physically stable. In this project, we study the use of LQR controller for balancing the robot, and extend the application for the robot to traverse any path. A differential drive mechanism is used while developing the robot, which helps in decoupling the translation and rotation stages in path traversal. An A* algorithm has been used for the path planning in a static obstacle occupied environment. It is shown that a LQR controlled robot can successfully help the robot stabilize on spot without falling, as well as enable linear motion in the forward direction.

Acknowledgements

I take this opportunity to express my deep gratitude towards my mentors for this research project Prof. Avinash Gautam and Prof. Sudeept Mohan for giving me this opportunity to work on this brilliant project in my domain of interest. I have learned a lot of things since the start of this project all because of this opportunity and sirs' guidance. I really felt at ease after every interaction I had with them. Through this course on Autonomous Mobile Robotics, I have been taught the required resources to develop an in-depth understanding of the basic concepts required to kickstart the project. I would also like to thank my friends and family members for adjusting their lifestyle to provide me with an effective work environment which ensures seamless attention and is free of any disturbances which was integral throughout the project. Finally, I express my sincere appreciation to all the Open Source projects and libraries without which advanced research in robotics wouldn't be possible.

Contents

Abstract i

Acknowledgements ii

Contents iii

List of Figures iv

1 Introduction 1

2 Robot Model and Kinematics 2

3 Sensors 5
- 3.1 Wheel Encoder . . . 5
- 3.2 Gyroscope . . . 6
- 3.3 Accelerometer . . . 7
- 3.4 Inertial Measurment Unit . . . 7
- 3.5 GPS . . . 8

4 LQR Controller 10

5 Path Planner 13
- 5.1 Map . . . 13
- 5.2 Planning Algorithm . . . 14

6 Conclusion and Future Work 18

A Code 20
- A.1 LQR . . . 20
- A.2 A* . . . 20

List of Figures

2.1 Robot Model 2
2.2 Kinematics of Differential Drive robot [3] 3

3.1 Gyroscope [4] 6
3.2 Accelerometer [2] 7
3.3 GPS [5] 8

5.1 Robot Arena 14
5.2 (a) Moore's neighborhood (b) Von Neumann neighborhood 14
5.3 A* Pseudocode [1] 15
5.4 A* Path using Von Neumann's neighborhood, Manhattan and Euclidean heuristic 16
5.5 A* Planned Path using Moore's neighborhood and Euclidean heuristic 16

Chapter 1

Introduction

Mobile robots have become commercially available for personal, industrial, military, hospital, and school applications. A critical design of these robots is to ensure they are physically stable. One type of robot that defies the usual convention of simple stabilization is one only having two wheels. With the recent push of autonomous robotics and intelligent vehicles, the self-balancing Two-Wheeled robotic system has become a helpful platform for larger, more complex robotic systems.

From a theoretical view, the problem of achieving self-balance on a two-wheeled system possess the characteristics of a high order, non-linear, multi-variable and unstable system. Control theorists and engineers have attracted much attention for its use as an experimental testbed in proving theorems, methods, and educational tools. The two-wheeled self-balancing problem is analogous to the Single Rotational Inverted Pendulum study, provided the robot is designed with a very low centre of gravity but with a heightened chassis. Controlling self-balancing robots is considered one of the most popular benchmark studies in non-linear control theory. Thus, much of the existing literature and theorems from the inverted Pendulum studies can be appropriately transferred to the two-wheeled balancing problem.

In this project, a cylindrical chassis of a differential drive robot is used with two standard wheels. The wheels are attached to the body and in contact with the ground. The body does not touch the ground. The robot needs to balance its body angle to stay upright to avoid failing. At the same time, the wheels' position should remain at the same spot. Furthermore, the balancing robot should be able to move to the front and back after all these movements have been done. The balancing robot leans to the front, and the wheel stays in a range. In this project, a balancing robot with Linear Quadratic Regulator (LQR) controller is used for stabilizing the system and helping the system achieve a specific state. The robot traverses a path in a static obstacle environment, where the path was found using the A* algorithm. The whole simulation was run using Webots software, and the code was developed using Python.

Chapter 2

Robot Model and Kinematics

The robot model is customized using the functionalities offered by the Webots simulation software. As seen in Figure 2.1, The robot has a cylindrical chassis of radius 0.045 units and a height of 0.08 units. Wheels of radius 0.025 units and width 0.01 units are attached to the chassis on the sides. It is a differential drive robot. There are two motors to control the two wheels independently—this enabled in-place rotation of the robot. The ground clearance of the robot (the chassis) is 0.0665 units. The exact structure of the robot model in terms of software (.PROTO file in Webots) can be seen in Figure 2.1.

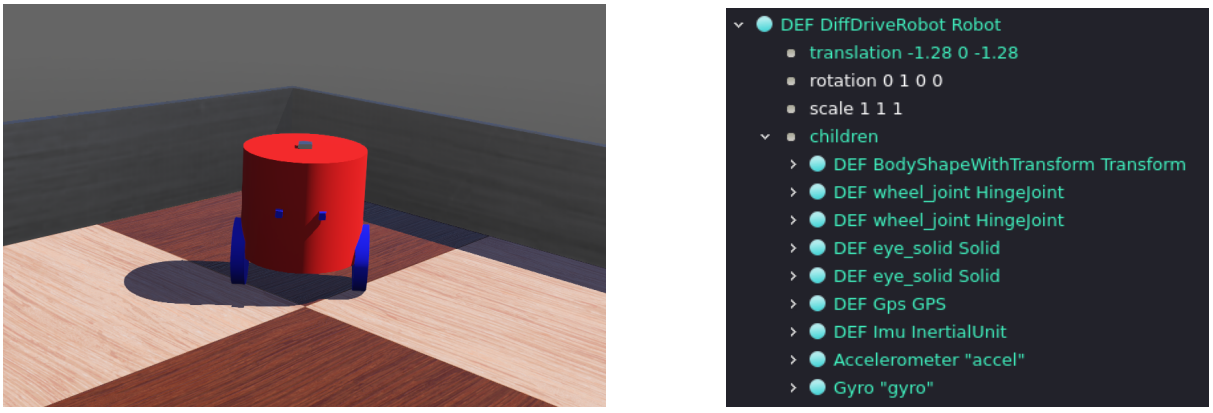

FIGURE 2.1: Robot Model

The wheels of a robot play an important part not only in locomotion but also in supporting the weight of the entire robot. The diameter of the wheels should be chosen in order to meet the torque and response requirements of the system. The net torque, which the motors exert on the wheels, is expressed as, $\tau = I \cdot \alpha$. Moment of inertia of a wheel with radius r, mass m is given by $I = M \cdot r^2$. Hence, $\alpha = \frac{\tau}{M \cdot r^2}$. For a given torque, an increase in the wheel diameter decreases the angular acceleration, slowing down the tilting of the robot, hence making it more stable against offsets and giving the motors more recovery time to balance it.

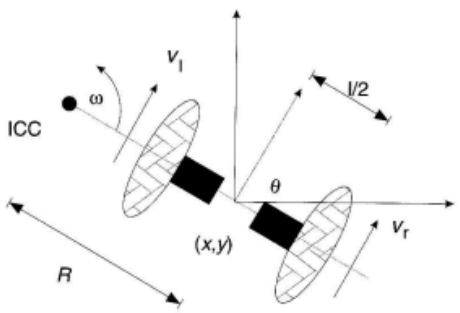

FIGURE 2.2: Kinematics of Differential Drive robot [3]

If a is the linear acceleration, then $a = \frac{\tau}{M \cdot r}$ An increase in diameter also slows down the linear acceleration, effectively decreasing the response time of motors to move quickly in the direction of the robot falling, in order to balance it. Thus, the selection of diameter of the wheel is a trade-off between the stability and quick response of the robot.

A differential drive robot consists of 2 drive wheels mounted on a common axis, and each wheel can independently being driven either forward or backward. While the velocity of each wheel can be varied, for the robot to perform rolling motion, the robot must rotate about a point that lies along their common left and right wheel axis. The point that the robot rotates about is known as the ICC - Instantaneous Center of Curvature.

By varying the velocities of the two wheels, the trajectories that the robot takes can be easily varied. Because the rate of rotation ω about the ICC must be the same for both wheels, following equations can be derived:

$$\omega \cdot (R + \frac{l}{2}) = V_r$$

$$\omega \cdot (R - \frac{l}{2}) = V_l$$

where l is the distance between the centers of the two wheels, V_r , V_l are the right and left wheel velocities along the ground, and R is the signed distance from the ICC to the midpoint between the wheels. At any instance in time R and ω can be found out using the following equations:

$$R = \frac{l}{2} \cdot \frac{V_r + V_l}{V_r - V_l}$$

$$\omega = \frac{V_r - V_l}{2}$$

There are three cases with these kinds of drives:

1. If $V_l = V_r$, robot moves forward performing linear motion. R becomes infinite, and there is no rotation as ω becomes zero.

2. Of $V_l = -V_r$, then $R = 0$, and the robot rotates in place about the midpoint of the wheel axis.

3. If $V_l = 0$, the robot rotates about the left wheel with $R = -\frac{l}{2}$. Similarly, if $V_r = 0$, the robot rotates about the right wheel with $R = \frac{l}{2}$

A differential drive robot cannot move in the direction along the axis - this is a singularity. Differential drive vehicles are very sensitive to slight changes in velocity in each of the wheels. Small errors in the relative velocities between the wheels can affect the robot trajectory. They are also very sensitive to small variations in the ground plane, and may need extra wheels (castor wheels) for support.

Chapter 3

Sensors

In order to control the robot, various sensors are needed to provide feedback. Controller needs the current position, orientation, as well as velocity as inputs. In the real world, every sensor has some type of drawback, or noise/bias/drift associated with its readings. Since this project aims to build a self-balancing robot in the simulation stage, perfect sensors without noise or drift are assumed. The robot needs the following sensors for the project:

3.1 Wheel Encoder

Wheel Encoders are a type of position sensor. They are of high quality, low cost, and have excellent resolution. Each encoder is located behind the motor near the wheel. It is used to count the number of times the motor rotated, based on which the robot's distance has driven or turned can be calculated. Because these sensors are proprioceptive, their position estimate is best in the robot's reference frame. When applied to the problem of robot localization, significant corrections are required.

As with most proprioceptive sensors, encoders are generally in the controlled environment of a mobile robot's internal structure, so systematic error and cross-sensitivity can be engineered away. The accuracy of optical encoders is often assumed to be 100%. Although this may not be entirely correct, any errors at the level of an optical encoder are dwarfed by errors downstream of the motor shaft.

The wheel encoder counts can be used to perform several useful robot behaviours:

1. The robot can drive straight lines by making minor adjustments in the left and right motor powers to ensure both motors rotate at the same average speed.

2. The robot can drive for a specific distance by calculating how far the wheels have travelled. This is combined with adjusting the motor powers to drive straight.
3. The robot can pivot on both wheels by a specific angle by calculating how far the wheels have travelled while pivoting in a circle.
4. The robot can turn on one wheel by a specific angle by calculating how far the driving wheel has travelled while turning in a circle.

Based on the size of the robot's wheels, the distance that the robot travels during one wheel revolution can be calculated. This distance is equal to the wheel's circumference (i.e., the distance around the outer edge of the wheel). This information can convert any encoder count into distance travelled or convert a desired distance into a target encoder count [6].

In Webots, the wheel encoders are programmed such that it has an attribute for noise and resolution. For the sake of simplicity, this project assumes zero noise. HingeJoints are used to connect the wheels to the chassis. Hence, the values returned by the encoders are in Radian (as programmed in Webots).

3.2 Gyroscope

A gyroscope is an inertial sensor that measures an object's angular rate with respect to an inertial reference frame. Conceptually, it is a spinning wheel in which the axis of rotation is free to assume any possible orientation. When rotating, the orientation of this axis remains unaffected by tilting or rotation of the mounting, according to the conservation of angular momentum. Due to this principle, a gyroscope can lead to the measurement of orientation and its rate of change.

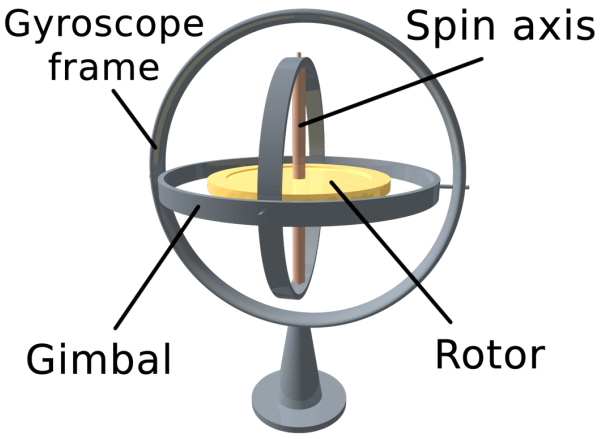

FIGURE 3.1: Gyroscope [4]

In Webots, gyroscope returns the angular velocity along the 3 axes. The angular velocity is measured in radians per second [rad/s]. The sensor is placed exactly along the line joining the

center of gravity and parallel to the wheel axles, at an altitude of 0.03325 meters. The angular rate measured by the gyroscope is not affected by the placement of the sensor, as it will vary by the same amount at all points on the robot. However, as the distance from the ground increases, linear acceleration and vibrations, measured at higher points, increases. This could lead to false readings on the accelerometer. Therefore, the sensor is placed at the bottom.

3.3 Accelerometer

An accelerometer is a primary sensor responsible for measuring inertial acceleration, or the change in velocity over time. It can be found in various types, including mechanical accelerometers, quartz accelerometers, and MEMS accelerometers. A MEMS accelerometer is essentially a mass suspended by a spring. The mass is known as the proof mass, and the direction that the mass is allowed to move is known as the sensitivity axis.

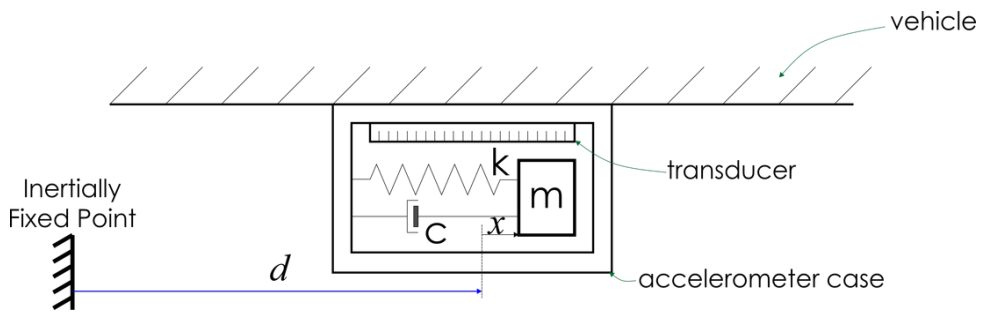

FIGURE 3.2: Accelerometer [2]

When an accelerometer is subjected to a linear acceleration along the sensitivity axis, the acceleration causes the proof mass to shift to one side, with the deflection proportional to the acceleration. In Webots, the accelerometer node measures acceleration and gravity induced reaction forces along the three axes. The sensor is placed exactly along the line joining the center of gravity and parallel to the wheel axles, at an altitude of 0.03325 meters.

3.4 Inertial Measurment Unit

An IMU (Inertial Measurement Unit) sensor is used to determine the motion, orientation, and heading of the robot. An IMU sensor is a complete package that includes an accelerometer, a gyroscope, a magnetometer, and sometimes a barometer sensor. Each one of the measures is represented in a three-axis coordinate system, so generally speaking, they both together yield a 6-dimension measurement time series stream.

1. Accelerometer is a sensor that measures the specific force (the body mass normalizes the force). It provides the acceleration across the x, y, and z axes in its local frame.

2. Gyroscope is a sensor that measures angular velocity around the x, y, and z axes, in its local frame. Generally, integrating the measurements results in the angles themselves.

3. Magnetometer is a sensor that measures the Earth's magnetic field and provides the heading (the compass is one such device). If it is included in the IMU, we commonly describe it as a "9-axis IMU."

4. Barometer is a sensor that measures air pressure and can provide altitude.

In Webots, the IMU returns the roll, pitch and yaw values only. Hence, for obtaining the angular velocities and the acceleration, gyroscope and an accelerometer have been used along with the IMU. They are redundunt while being used in the real world. In the robot, the IMU is placed exactly along the line joining the center of gravity and parallel to the wheel axles, at an altitude of 0.03325 meters.

3.5 GPS

The Global Positioning System (GPS) is a space-based radio-navigation system consisting of a constellation of satellites broadcasting navigation signals and a network of ground stations and satellite control stations used for monitoring and control. Currently, 31 GPS satellites orbit the Earth at an altitude of approximately 11,000 miles providing users with accurate information on the position, velocity, and time anywhere in the world and in all weather conditions.

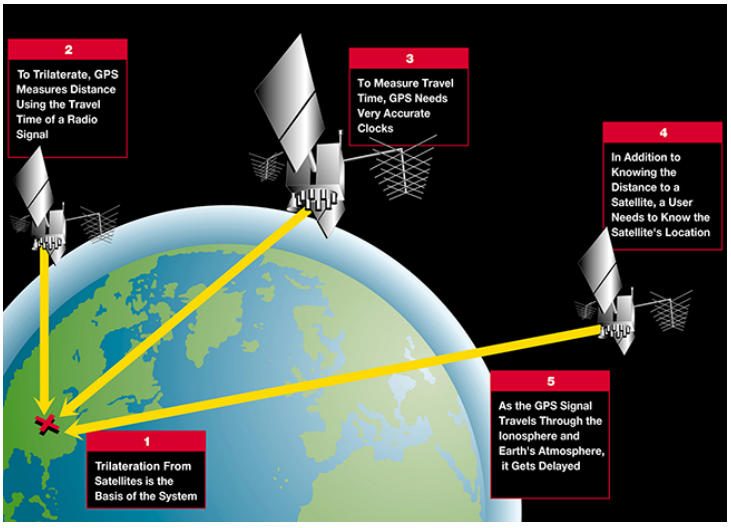

FIGURE 3.3: GPS [5]

The basic GPS service provides users with approximately 7.0-meter accuracy, 95% of the time. To accomplish this, each of the 31 satellites emits signals that enable receivers to determine their location and time through a combination of signals from at least four satellites. GPS

satellites carry atomic clocks that provide highly accurate time. The time information is placed in the codes broadcast by the satellite so that a receiver can continuously determine the time the signal was broadcast. The signal contains data that a receiver uses to compute the satellites' locations and make other adjustments needed for accurate positioning. The receiver uses the time difference between the time of signal reception and the broadcast time to compute the distance, or range, from the receiver to the satellite. The receiver must account for propagation delays or decreases in the signal's speed caused by the ionosphere and the troposphere. The receiver can compute its three-dimensional position with information about the ranges to three satellites and the satellite's location when the signal was sent. An atomic clock synchronized to GPS is required to compute ranges from these three signals. However, by reading a measurement from a fourth satellite, the receiver avoids the need for an atomic clock. Thus, the receiver uses four satellites to compute latitude, longitude, altitude, and time [6].

In this project, the GPS has been used to provide the feedback of the robot position, and thus calculate the error distance the robot needs to travel in order to reach its waypoint in the path. The GPS is fixed on top of the robot with an altitude of 0.105 meters.

Chapter 4

LQR Controller

Linear Quadratic Regulator, or LQR, is a commonly used algorithm for obtaining optimal control. It is based on the fundamentals of the optimization problem, which means maximizing or minimizing a function subject to some constraints on the variables. The constraints can be equality or inequality constraints based on the problem. The function to be maximized is the cost function, given by the dynamics of the environment, particularly the robot. This function gives the next state of the robot based on the current action and the current state. This is similar to the problem defined in reinforcement learning.

State of the robot: A state-space consists of all the variables which are required to completely describe a robotic system from one timestep to the next. These variables include the current position (e.g. X, Y, Z coordinates), orientation (roll pitch yaw, or quaternion), the velocity (angular and linear), and even the acceleration (angular and linear) of the robot in the world. It is up to the user to decide what all is needed in the state space for the user's application. For example, in a 2D system, the Z-axis variables, roll, and pitch will always be 0 (ideally). Depending on the degree of freedom of the robot, more variables can be ignored. In this project, the robot is programmed to move forward or turn in place. It cannot do both of them simultaneously. Hence, the state space further simplifies, ignoring the Y-axis variables. So the resulting state space consists of the position coordinate x, velocity in the X direction, yaw (θ), and the rate of change of yaw ($\dot{\theta}$).

$$X = [x \quad \dot{x} \quad \theta \quad \dot{\theta}]^T$$

$$U = -K \cdot X$$

In the above equation, X is the state variable at each time step, U is the action. Static gain matrix K denotes the parameters of the LQR. It is essentially a matrix multiplication, with each gain in K corresponding to each state variable in X. There are many mathematical equations

to derive the K matrix of the LQR controller. For maintaining the simplicity of the project, K is found empirically.

The position and the yaw inputs of the controller are obtained by reading the sensor data. An Accelerometer, Gyroscope, and wheel encoders are used for the same. The wheel encoders supply the position data, while the angle θ is obtained from the gyroscope and the accelerometer. An accelerometer measures all forces/accelerations on the object including the forces that drive the system (motor). So, in motion, it is affected by acute noise (vibrations) which may give us false angle computations even when there is no rotation (High frequency noise). It also has a slow response time. Whereas, in a gyroscope, measurement is not susceptible to external forces, but it has tendency to drift, not returning to zero when the system comes back to original position, which has a cumulative effect, and thus, is accurate for short term measurements (Low frequency noise). It has a faster response time. A **complimentary filter** is used to remove both of these noises. A low pass filter and a high pass filter is combined to fuse the pitch angles calculated by the accelerometer and gyroscope readings as follows:

$$\theta = \alpha \cdot gyro_rate \cdot \dot{t} + (1 - \alpha) \cdot accel_angle$$

In this project, the LQR controller is used to calculate the control inputs for the robot. This would have meant inputs to the motors controlling the wheel in the real world. Webots simplifies this by giving the user access to directly set the speed of the wheels using the *setVelocity* function of the motor class. The LQR function is defined here such that it directly acts on the error of the state space variables relative to their setpoints. This completes the closed loop control.

The controller works such that the robot turns *in place* and attains the orientation required along the edge joining two consecutive waypoints in the path. The error in angle θ is calculated as the difference between the required orientation and the current yaw of the robot. Both are obtained with respect to the world frame. The required turning angle is calculated using the *Atan2* function, taking the x and z coordinates of the node as input. The current yaw is obtained by reading the IMU sensor data. A threshold of 0.005 radian is set for the error below which the robot stops turning. After that, *forward motion* is activated and the robot moves along the edge. A threshold of 0.02 meters is set for the error below which the waypoint is popped out of the path, and the robot starts to approach the next waypoint. If the path is completely traversed, the robot just balances itself at the goal orientation. The controller does not stop running, as that will cause the robot to fall down. The Python implementation of the simple LQR controller is provided in the appendix A.

One of the drawbacks of using this single loop control system is that the robot's actions take a lot of time to complete. So the time needed to traverse a path is a lot. Another factor behind this is that the controller needs to balance the robot and prevent it from falling down, as well

as make the robot turn in place or move it in a forward direction. A more complex control architecture will solve these problems.

Chapter 5

Path Planner

Path-planning is an important primitive for autonomous mobile robots that lets robots find the shortest – or otherwise optimal – path between two points. Otherwise optimal paths could be paths that minimize the amount of turning, the amount of braking or whatever a specific application requires. In this project, the path is optimized in terms of the distance travelled by the path.

5.1 Map

Path-planning requires a map of the environment and the robot to be aware of its location with respect to the map. Map representations can be discrete or continuous approximations of the real world. In a discrete approximation, a map is sub-divided into sub-parts of equal or differing sizes. A continuous approximation requires the definition of inner (obstacles) and outer boundaries, typically in the form of a polygon, whereas paths can be encoded as sequences of real numbers. Discrete maps are the dominant representation in robotics.

This project uses a grid based map representation, which is a type of equal sized discrete map. Discrete maps can be said to be a graph representation, with every vertex of the map corresponding to a node of the graph, which are connected by edges. The robot can navigate from one vertex to the other over the edge. The edges are decided by the number of neighbors considered over any given cell/node in the grid. There are two types of neighborhood- Moore's and Von Neumann's neigborhood. This can be seen in the figure 5.2.

Computationally, a graph might be stored as an adjacency or incidence list/matrix, or even using a user defined class. This project uses a list/matrix representation for the map. The grid can be seen as a binary occupancy grid, 0s denoting free space, and 1s denoting obstacle occupied space. In a grid map, the environment is broken down into discrete squares of arbitrary resolution,

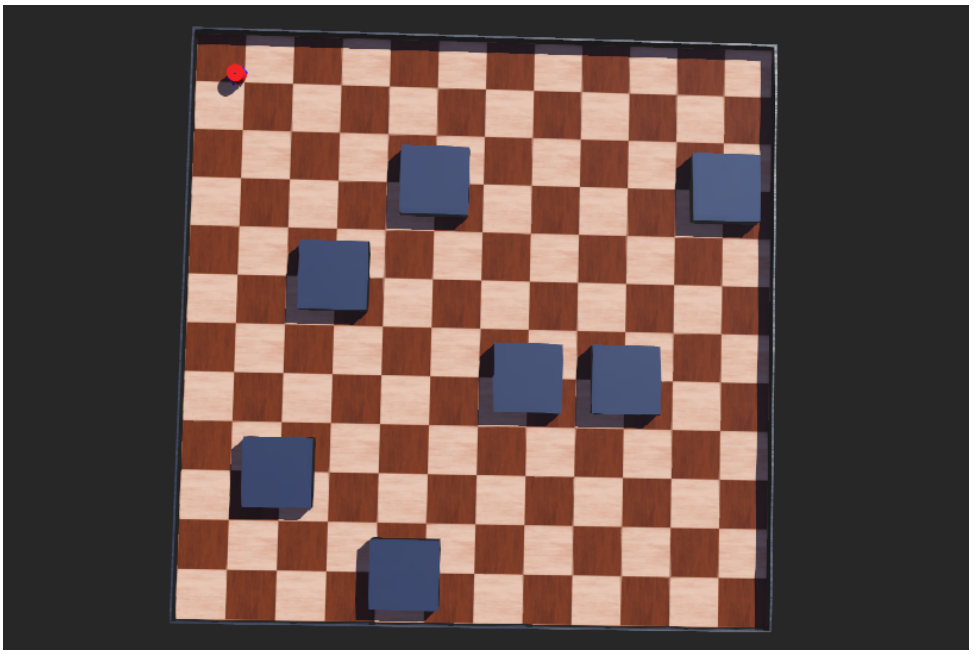

FIGURE 5.1: Robot Arena

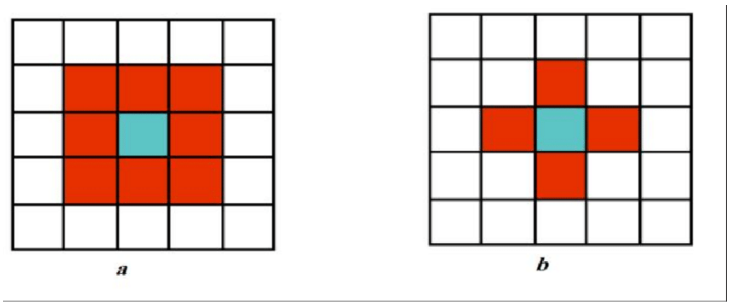

FIGURE 5.2: (a) Moore's neighborhood (b) Von Neumann neighborhood

depending on the size of the obstacle, and the fineness needed by the robot in traversing the path. Disadvantages of grid maps are their large memory requirements as well as computational time to traverse data structures with large numbers of vertices. In this project, a very small scale environment is considered, so the disadvantage does not cause concerns.

5.2 Planning Algorithm

The goal of path planning is to find the best path in the map's connectivity graph between the start and the goal, where best refers to the selected optimization criteria (e.g., the shortest path). In this project, A-Star algorithm has been used for finding the shortest path. A* Before diving in the algorithm, it is essential to discuss the discriminators and the functions used in the algorithm:

1. Total cost $f(n)$

2. Path cost $g(n)$
3. Edge traversal cost $c(n, n')$
4. Heuristic cost $h(n)$

Here, n is the current node, and n' is the adjacent node. The accumulated cost from the start node to any given node n with $g(n)$. The cost from a node n to an adjacent node n' becomes $c(n, n')$, and the expected cost (heuristic cost) from a node n to the goal node is described with $h(n)$. The total expected cost from start to goal via state n can then be written as

$$f(n) = g(n) + \epsilon \cdot h(n)$$

where ϵ is a parameter that assumes algorithm-dependent values. ϵ is the weight given by the algorithm to the heuristic cost $h(n)$. For the A* algorithm, the value of ϵ is 0.

The goal node is denoted by node_goal and the source node is denoted by node_start

We maintain two lists: **OPEN** and **CLOSE**:

OPEN consists on nodes that have been visited but not expanded (meaning that sucessors have not been explored yet). This is the list of pending tasks.

CLOSE consists on nodes that have been visited *and* expanded (sucessors have been explored already and included in the open list, if this was the case).

```
1  Put node_start in the OPEN list with f(node_start) = h(node_start) (initialization)
2  while the OPEN list is not empty {
3    Take from the open list the node node_current with the lowest
4        f(node_current) = g(node_current) + h(node_current)
5    if node_current is node_goal we have found the solution; break
6    Generate each state node_successor that come after node_current
7    for each node_successor of node_current {
8      Set successor_current_cost = g(node_current) + w(node_current, node_successor)
9      if node_successor is in the OPEN list {
10       if g(node_successor) ≤ successor_current_cost continue (to line 20)
11     } else if node_successor is in the CLOSED list {
12       if g(node_successor) ≤ successor_current_cost continue (to line 20)
13       Move node_successor from the CLOSED list to the OPEN list
14     } else {
15       Add node_successor to the OPEN list
16       Set h(node_successor) to be the heuristic distance to node_goal
17     }
18     Set g(node_successor) = successor_current_cost
19     Set the parent of node_successor to node_current
20   }
21   Add node_current to the CLOSED list
22 }
23 if(node_current != node_goal) exit with error (the OPEN list is empty)
```

FIGURE 5.3: A* Pseudocode [1]

A* works by making a lowest-cost path tree from the start node to the target node. A* algorithm begins at the start (yellow star), and considers all adjacent cells. Once the list of adjacent cells has been populated, it filters out those which are inaccessible (walls, obstacles, out of bounds). It then picks the cell with the lowest cost, which is the estimated $f(n)$. This process is recursively repeated until the shortest path has been found to the target (red star). The computation of

FIGURE 5.4: A* Path using Von Neumann's neighborhood, Manhattan and Euclidean heuristic

$f(n)$ is done via a heuristic that usually gives good results. A* differs from an algorithm in that a heuristic is more of an estimate and is not necessarily provably correct.

Famous examples of heuristics are the "Euclidean" and the "Manhattan" distance. The heuristic function must be *admissible*, which means it can never overestimate the cost to reach the goal. The value of $h(n)$ would ideally equal the exact cost of reaching the destination. This is, however, not possible because the best path is not known. The Manhattan distance is usually preferred for grid based map representations, with Von Neumann's neighborhood (only four neighbours at right angles). It is given by $h(n) = |n_x - goal_x| + |n_y - goal_y|$. This heuristic is exact whenever our path follows a straight lines. The paths obtained by using Von Neumann's neighborhood along with Manhattan and euclidean heuristics can be seen in figure 5.3.

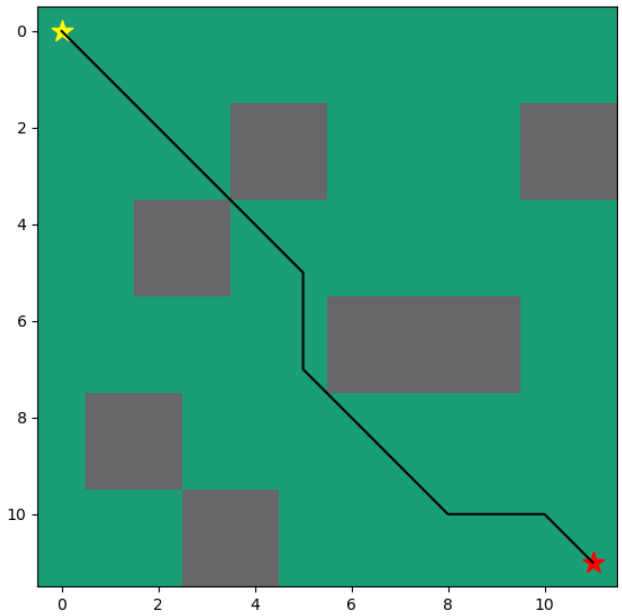

FIGURE 5.5: A* Planned Path using Moore's neighborhood and Euclidean heuristic

The optimal path is given when Moore's neighborhood (8 neighbors) is considered. In this case, Euclidean and Manhattan heuristics give the same results. The path can be seen in figure 5.4. An important point to note is that a buffer distance of 0.15 meters is considered around the static

obstacles. This takes into account the size of the robot, as well as the possible deviations the robot might make while traversing the path. The deviations might be caused by the inaccuracies in sensor readings, increasing bias, or addition of any kind of noise. A solution for this is to use some sort of a filter for doing sensor fusion. This project does not consider any noises in the sensor readings, and hence no filter has been used.

The Python implementation of the A* path planner is provided in the appendix A.

Chapter 6

Conclusion and Future Work

A strictly two-wheeled robot (without a free castor wheel for support) provides a helpful platform for testing complex control algorithms. It helps ensure if a multi-variable system will be physically stable. The project illustrates the use of LQR controller for stabilizing a two wheeled robot. It extends on-spot balancing of the robot to make the robot traverse any path successfully. A* algorithm is used to successfully find the shortest path in an environment consisting of static obstacles. Different kinds of heuristics are experimented with, and it was seen that the number of neighbours (Moore's and Von Neumann's neighborhood) considered for path finding rendered the heuristic ineffective in some cases.

Future work involves developing a more robust and multi-loop control architecture, which will speed up the process of path traversal. Dynamic obstacle avoidance can be implemented with the help of sensors like camera or LiDAR.

Bibliography

[1] *A* Algorithm Pseudocode*. Lluís Alsedà. URL: https://mat.uab.cat/~alseda/MasterOpt/AStar-Algorithm.pdf.

[2] *Accelerometer*. URL: https://www.autonomousrobotslab.com/inertial-sensors.html.

[3] *Differential Drive Kinematics*. Dudek and Jenkin. URL: https://www.cs.columbia.edu/~allen/F19/NOTES/icckinematics.pdf.

[4] *Gyroscope*. URL: https://en.wikipedia.org/wiki/Gyroscope (visited on 2021).

[5] *Satellite Navigation - GPS - How It Works*. URL: https://www.faa.gov/about/office_org/headquarters_offices/ato/service_units/techops/navservices/gnss/gps/howitworks.

[6] Roland Siegwart, Illah Reza. Nourbakhsh, and Davide Scaramuzza. *Introduction to autonomous mobile robots*. MIT Press, 2011.

Appendix A

Code

A.1 LQR

```
def lqrSpeed(x, dx, theta, theta_dot):
    state[0] = x
    state[1] = dx
    state[2] = theta
    state[3] = theta_dot

    d = [K[i]*state[i] for i in range(len(K))]
    s = sum(d)

    if abs(s) > max_spot_speed:
        s = max_spot_speed*(s/abs(s))
    return s
```

A.2 A*

```
import heapq

def heuristic(a, b, type = "euclidean"):
        if type == "euclidean":
            return np.sqrt((b[0] - a[0]) ** 2 + (b[1] - a[1]) ** 2)
        elif type == "manhattan":
                return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar(array, start, goal):

        neighbors = [(0,1),(0,-1),(1,0),(-1,0),(1,1),(1,-1),(-1,1),(-1,-1)]
        close_set = set()
        came_from = {}
        gscore = {start:0}
        fscore = {start:heuristic(start, goal)}
        oheap = []
```

```
heapq.heappush(oheap, (fscore[start], start))

while oheap:

        current = heapq.heappop(oheap)[1]
        if current == goal:
                data = []
                while current in came_from:
                        data.append(current)
                        current = came_from[current]
                return data
        close_set.add(current)
        for i, j in neighbors:
                neighbor = current[0] + i, current[1] + j
                tentative_g_score = gscore[current] + heuristic(current, neighbor)
                if 0 <= neighbor[0] < array.shape[0]:
                        if 0 <= neighbor[1] < array.shape[1]:
                                if array[neighbor[0]][neighbor[1]] == 1:
                                        continue
                        else:
            # array bound y walls
                                continue
                else:
                        # array bound x walls
                        continue

                if neighbor in close_set and tentative_g_score >= gscore.get(neighbor, 0):
                        continue

                if  tentative_g_score < gscore.get(neighbor, 0) or neighbor
                not in [i[1] for i in oheap]:
                        came_from[neighbor] = current
                        gscore[neighbor] = tentative_g_score
                        fscore[neighbor] = tentative_g_score + heuristic(neighbor, goal)
                        heapq.heappush(oheap, (fscore[neighbor], neighbor))

return False
```